IPv6 with Support of PineCone

Mrithu Bashini Bhalkrishnan <mb.bhalkrishnan@stud.fh-sm.de>, 319293
Monika Hosadurga Thippeswamy<thippesw@stud.hs-sm.de>, 319922
Ameerali Kudukkathu Valappil <a.kvalappil@stud.fh-sm.de>, 319959
Anshul Sharma <a.sharma@stud.fh-sm.de>, 319579
Purvika Panwar <p.panwar@stud.fh-sm.de>, 319301

Abstract

Internet of Things (IoT) technology has quickly developed as an essential component of our daily life, allowing various devices to be networked together from sensors to domestic appliances, wearables, and industrial devices. Almost all devices involved in the Internet of Things, including development boards such as PineCone BL602, were using IPv4 for networking. However, the address space of IPv4 is limited , which offers only 4 billion addresses. This is the major drawback of IPv4 so the internet is started shifting towards IPv6. A new Internet protocol with a much larger address space that is best for the network setups. This project tries to establish the two development boards, PineCone BL602, networking each other using IPv6 only, without any need of IPv4 or any external network setup. This is done by preparing one of the PineCone development boards to work as a Wi-Fi Access Point, and the other board to work as a Wi-Fi Station. Within this, a network is created that allows the two development boards to connect with each other using only IPv6 networking. Changes are necessary in the network configurations of both boards to support the IPv6 functionality, such as address autoconfiguration and Neighbor Discovery, within the Bouffalo SDK and the lwIP networking. Unlike other IoT projects that require additional hardware like routers or Raspberry Pi, this one does not require such hardware components. This is with the aim that the two PineCone boards, allowing communication between the two PineCone boards using IPv6 without the need for routers, to prove the possibility of IPv6 implementation on development boards from PineCone and Successful execution of this project indicates that the BL602 platform is capable of handling modern networking systems, which is important for any IoT network that requires efficient networking. This offers a low-cost and easy design makes it ideal for small IoT networks, research, and educational purposes for embedded networking.

Contents

1 Introduction **4**

2 Background **6**

3 Literature Review **7**

4 System Architecture **10**
4.1 Overview . . . 10
4.2 Hardware Architecture . . . 10
4.3 Software Architecture . . . 11
4.3.1 Operating System Layer . . . 11
4.3.2 Networking Stack Layer . . . 11
4.3.3 Wi-Fi Management subsystem . . . 12
4.3.4 Application Layer . . . 12
4.4 Communication Flow and Data Path . . . 12
4.5 Design Consideration and Constraints . . . 12

5 Implementation **13**
5.1 Development Setup and Firmware Structure . . . 13
5.2 lwIP IPv6 Enablement and Configuration . . . 13
5.3 Access Point Firmware Implementation . . . 14
5.3.1 AP Role and Responsibilities . . . 14
5.3.2 Wi-Fi AP Startup and the Deadlock Fix . . . 14
5.3.3 IPv6 Interface Bring-Up on AP . . . 15
5.4 UDP Echo Server (IPv6) . . . 15
5.5 Station Firmware Implementation . . . 15
5.5.1 STA Role and Responsibilities . . . 15
5.5.2 Connection State Handling and IPv4 Dependency (Previous Work Applied) . . . 16

6 Methodology **17**
6.1 Research Methodology Overview . . . 17
6.2 Layered Validation Approach . . . 18
6.3 Diagnostic Tools and Techniques . . . 19

7 Debugging and Problem Solving **19**
7.1 Phase 1: System Instability Due to Heap Exhaustion . . . 19
7.2 Phase 2: Wi-Fi Access Point Deadlock . . . 20
7.3 Phase 3: Station Disconnection Due to IPv4 Dependency . . . 20
7.4 Phase 4: IPv6 UDP Routing Failure . . . 20
7.5 Engineering Insights from the Debugging Process . . . 21

8 Results and Evaluation **21**

9 Discussion **22**

10 Future Scope **23**
10.1 Advanced IPv6 Routing and Neighbor Discovery Analysis 23
10.2 Deeper Protocol-Level Tracing Beyond UART Logging 24
10.3 Investigation of Interface Binding and Address Scope Handling 24
10.4 Removal of IPv4 Dependency in the Wi-Fi Manager 24
10.5 Driver-Level Verification of IPv6 Frame Transmission 24

11 Conclusion **25**

12 Review and Remarks from Authors of Other Teams **26**

List of Tables

1 Reviewer Comments and Author Responses 26

List of Figures

1 System architecture diagram. 11
2 Ap Starting Point. 14
3 AP creating the IPv6 address. 15
4 Starting point of STA where the IPv6 is connected. 16
5 Data Packet issue. 17
6 Work Flow of the system. 18
7 Laptop connected with AP and IPv6 is Assigned. 22

1 Introduction

Small wireless devices are beginning to emerge in nearly everything in the last several years. We use whether it is a simple temperature sensor at home or a more intelligent automated one systems in industries. Each of these devices is based on a network layer which assists them to share information with each other. With more devices becoming connected to the IoT system, there is a growing influence on the system connected, the communication between these devices turns to be as significant as a hardware they run on. Due to this, there is a growing concern in finding out the way. The internet protocols operate with small power boards which were initially built to accommodate very simple tasks. Each device needs a unique IP address, as do each household must have a separate mailing address. Devices were long used to operate on IPv4. IPv4 has been the most heavily used protocol used to interconnect devices on the internet over years. It has 4.3 billion unique addresses provided by the 32-bit addresses. In the beginning, it seemed such as this would be sufficient, and as more and more devices become interconnected, we are becoming exhausted of addresses. To address this issue, such features as Network Address Translation(NAT) have to be fixed. The Network Address Translation(NAT) enables a large group of devices on a network to have the use of a single IPv4 address. Even though it will assist in the issue of the out of running of addresses, it complicates the matter and it does not allow device-to-device communication. This is an issue with IoT, because it involves additional devices-to-devices communication and reduced dependency on central systems. IPv6 was created to take the place of IPv4 and eliminate the problems associated with it. It was meant to give the internet space to expand. It has 128-bit addresses giving a very large number of unique addresses to devices, 3.4×10^{38}. Besides the improvement in the number of addresses, IPv6 has design networking that is easier and efficient. It is one of the significant concepts of IPv6 which is known as Stateless Address Auto-configuration(SLAAC). With SLAAC, devices can be granted the ability to communicate with one another they do not need a server to address their own IPv6 like IPv4 which has a server known as DHCP(Dynamic HostConfiguration Protocol). When a device having an IPv6 interface it comes online, and it automatically forms a link local IPv6 address, using information obtained, acquired in the interface, typically by the Media access control(MAC) address of the interface. SLAAC is very useful in the development of IoT networks since the devices are capable of configuring themselves automatically without requiring manual configuration. One of the features of IPv6 is the Neighbor Discovery Protocol (NDP), a replacement of the IPv4 Address Resolution Protocol(ARP) and provides the possibility of finding neighboring IP address, addresses of link-layers and the state of reachability with the help of ICMPv6 messages. These are everything to make the network flow and communication easier especially in a decentralized network architecture. It has also made it more secure with the requirement to support the Internet Protocol Security (IPsec) of secure data communication in the network. All these are excellent aspects of a network protocol such as IPv6 to the IoT. Regardless of these benefits, IPv6 usage in embedded systems remains restricted at the moment as compared to current to its usage in computers. This is because a large majority of the low-cost IoT platforms are yet to be communicating in the network with IPv4, which is only restricted

in firmware, hardware, and network stack compatibility. Embedded systems are small in terms of memory, processing and energy consumption capacity. Embedded systems are difficult to implement with IPv6 and can introduce system instabilities due to the increased header length that it uses, more protocol structures and more functions such as NDP and ICMPv6. In addition, The embedded system network stacks are both IPv4 based and need major alterations in the system, and network hardware drivers, to enable IPv6. PineCone BL602 PineCone BL602 is an IoT development board product, built upon the RISC-V, which is also a cost effective product is an architecture and has two features Wi-Fi and Bluetooth connectivity. It is a good option to conduct research and development since it is cost effective and have wireless ideas. However, the IPv6 is not available in the current state of the software tools. It is possible in theory for the hardware to support modern networking technologies, and it is in practice case, not so easy due to the manner in which the firmware is designed and available resources. The analysis of IPv6 implementation on this board can help us to understand the possible difficulties and troubles in the system. One of the challenges of researchers on embedded networking will be to prove that the existing Internet protocols like IPv6 can run on small resource limited machines without extra infrastructure. Many IoT systems use more infrastructure including external routers, gateways or even a cloud connection, to enable communication between devices. There are some cases, such as remote monitoring, industry BU, or even sensor networks, devices may be required to use a direct network with each other without any further networking infrastructure. Evidence of device-to-device communication using IPv6 in an embedded networking context is a significant move towards the development of IoT systems. The first thought behind this project is to determine whether it is possible, two boards of PineCone BL602 to communicate with each other using IPv6 when they is fully enclosed in a wireless network with no routers or connection to the Internet. In this network, one of the boards will be programmed as a Wi-Fi Access Point and the other board will be programmed as a Wi-Fi Station, and thereby form a linkage between these two boards. This project also has the advantage of enabling the communication of these boards to be possible chances to resolve some of the larger problems that occur in such cases, (e.g. memory constraints) Lack of timing and synchronization problems in event-based code, the way the Wi-Fi drivers operate with the networking stack and problems with routing in such protocols, one should be aware of these challenges so that embedded networking can be made reliable and scalable. Even though the complete communication is not captured, the debugging and analysis offer some useful knowledge which could be applied in future research development and created. Besides the technologies, this project is convenient in regards to learning us on how IPv6 really works. The use of IPv6 implementation. on a small IoT board gives us an application of mechanisms like in the real world. And link-local addresses, automatical configuration and neighbour discovery which are essential in link-local addresses, autoconfiguration and neighbour discovery are also essential in a case in which a group of internet of things applications are shifting to IPv6. Problems such as memory deficiency, network building and necessity of a debugging tool are commonly felt out there. This study can be regarded as giving useful information on how modern networking protocols may be implemented in resource constrained embedded devices. This is because it does not only tell what can and cannot be done when using

IPv6 in IoT devices in low-cost, however, it also addresses the significance of conducting system-level analysis with the networking protocol features.

2 Background

IPv6 is designed to solve the problems with IPv4 by using 128-bit addresses, which provide about 3.4×3^{38} unique addresses. In addition to this increased number of address space, IPv6 has other several benifits of design improvements such as a simplified header, auto-configuration, increased routing efficiency, support for multicasting, and improved security features. These are the characterstics that make IPv6 suitable for IoT devices since they can be automated and require minimal management on a daily basis. IPv6 is still less common in embedded devices as it is in computer networks. Many low-cost IoT development boards continue to still depend on IPv4 due to firmware constraints, hardware limitations, and compatibility issues within networking stacks. Embedded systems normally works under strict resource constraints, including limited memory, reduced processing power, and energy efficiency requirements. Implementing IPv6 on such platforms usually adds additional complexity because IPv6 requires larger headers, more protocol structures, and additional mechanisms such as Neighbor Discovery and ICMPv6 processing. These requirements increase memory consumption and computational overhead, which can create stability challenges for resource constrained devices. The embedded networking stacks are frequently optimized for IPv4 operation, and enabling IPv6 functionality may require deep modifications across multiple system layers, including the operating system, network stack configuration, and hardware drivers. One of the feature is the use of Stateless Address Autoconfiguration (SLAAC). It allows devices to configure their own addresses without the need for a server. It is very useful in IoT networks since manual configuration is very difficult. Devices use interface IDs based on hardware information such as the (Media Access Control) MAC address. Another important thing is the Neighbor Discovery Protocol (NDP), which is used in IPv4 networks in place of Address Resolution Protocol (ARP). It is used to locate neighbors, to obtain link-layer addresses, to keep track of which nodes are reachable, and to detect duplicates using ICMPv6 messages. In many cases, the firmware architecture of the embedded platform has been designed with IPv4 assumptions embedded in the networking subsystem. This means that, in many cases, the enabling of IPv6 requires changes to many parts of the system, including the operating system, the network stack configuration, and the device driver interfaces. Implementing IPv6 on embedded systems has its own set of challenges that are not usually found when dealing with normal computing systems. Embedded systems, such as the PineCone BL602, operate with very tight constraints on memory, processing power, and power consumption. Such constraints have a major impact on how networking is implemented. A major concern is related to memory consumption. It is notable that compared to IPv4, IPv6 has larger headers, additional protocol structures, and complex routing. Let's consider, Neighbor Discovery, ICMPv6, and multicast management all play a role in increasing memory consumption. Such consumption may lead to heap exhaustion. The hardware abstraction layers and wireless drivers are some of the factors that are making IPv6 integration harder. Most Wi-Fi firmware designs are created mostly for IPv4, and

this can cause problems, especially for IPv6 networks. This has forced developers to alter the firmware to accommodate both IPv4 and IPv6 networks. A well known open source TCP/IP stack is Lightweight IP (LwIP), which is especially developed to be used on embedded systems with low resources. It supports IPv4, IPv6, TCP, UDP, ICMP, and other services with minimal memory usage. lwIP has an architectural structure composed of various layers, each performing a unique function. The network interface layer (netif), which is responsible for interacting with hardware and handling network interfaces. The IP layer, which is responsible for routing and assigning IP addresses to devices connected to a network. The transport layer, which offers support for TCP and UDP8,h protocols to facilitate reliable and best-effort data transfer, respectively. The application programming interface (API), which provides sockets to applications to help communication.

3 Literature Review

Building this project stems from the basic fact that we are running out of IPv4 addresses. IPv4 utilizes 32 bits of information, which can have a maximum of approximately 4.3B unique addresses. It may seem like a large number, however, prior to the explosion of IoT devices, we were well on our way to exhausting the available IPv4 address space. IPv6 was created as a solution for this problem [3]. The IETF has already developed and published the formal specification for the new protocol, named RFC 8200. Among other things, IPv4 has been replaced by IPv6 and now utilizes a 128-bit address space, making it virtually impossible to exhaust the available address space in the near future. In addition to providing access to a much larger address space than is currently available via IPv4, IPv6 provides an improved overall architecture that should greatly benefit the entire community of people and businesses using the global Internet. IPv6 uses fixed length base headers therefore, parsing each packet requires far fewer processing resources than was previously required for IPv4 packets [4]. With SLAAC, devices self-configure their address, rather than having to go through a DHCP Server. Additionally, NDP replaces ARP for determining where to find other devices on the same link. Collectively, these are the significant architectural enhancements that makes IPV6 well suited for the modern networks in this age, not just a larger version of IPV4. Putting numbers in perspective is critical when discussing the scale of the problem. According to Ericsson's Mobility Report, over the next ten years there will be tens of billions of new IoT devices which will get connected. Using NAT has extended the lifespan of the IPv4 system by allowing multiple devices to use one IPv4 address. However, this also makes things more complicated because multiple devices cannot talk to each other directly, something that is critical to an IoT system. Another significant issue is that many platforms which could support IPv6 [4] in their networking stacks still come shipped with an IPv4-based configuration. One example of this is the BL602, where all of the IPv6 configuration flags in lwIP are set to off by default. Also, when the code for the Wi-Fi manager was written the thought of having IPv6 support was never given consideration. Therefore, in order for BL602 to run with IPv6 there were number of changes to be made to all your builds, configuration for the stacks, and the state machines handling the connections, it isn't just as simple as

flipping one switch. Wu, Cui, Wu, Liu, and Metz (2013) [8] provided a comprehensive survey of transition techniques from IPv4 to IPv6, including dual stack configurations, tunnelling approaches such as 6in4 and Teredo, and translation techniques like NAT64. Their largest contribution to this project is not about any one technique, but about where failures actually come from. The problem of failing to connect using IPv6 is generally not due to missing routing infrastructure. It tends to come from IPv4 assumptions built into firmware state machines, protocol implementations, and hardware abstraction layers. There are cases where a device has IPv6 compiled in but still does not behave as if it has a working IPv6 connection. This was directly relevant here. The Bouffalo SDK's [11] Wi-Fi manager only considers a device connected when a WIFI_EVENT_GOT_IP event is received. That event does not fire until an IPv4 address has been assigned. The result was that even when the IPv6 interface had fully come up with a valid link-local address and Neighbor Discovery working, the firmware still treated the device as disconnected and forced it off the network. The timing of these events lined up exactly with DHCP timeout windows. Wu et al. described this class of failure at the networking infrastructure level and this project ran into the exact same thing on a chip that costs a few euros. lwIP was first created by Adam Dunkels [6] at the Swedish Institute of Computer Science to fit a working TCP/IP stack into the limited RAM available on embedded systems, typically in the range of tens of kilobytes. lwIP achieves this using compile time flags. Each protocol feature is controlled by a setting in the lwipopts.h file, and anything turned off simply does not exist in the final binary, which matters a lot on constrained hardware. Gupta, Kumar, and Singh (2016) [7] specifically studied lwIP's IPv6 support on 32-bit embedded microcontrollers and their results matched what was found in this project. Setting LWIP_IPV6=1 alone does not produce a working IPv6 stack. It also requires LWIP_IPV6_ND6 for Neighbor Discovery, LWIP_ICMP6 for ICMPv6, LWIP_IPV6_AUTOCONFIG for SLAAC, and enough address slots configured per interface. Each of these modules brings its own runtime data structures at startup: Neighbor Cache Tables, Prefix Lists, Router Entries, and Address State Machines. On a constrained device, the total memory consumed by all of these together can easily exceed the normal heap allocation limit, and that is exactly what caused the problem documented in Section 8.1 of this report. Gupta et al. also noted that enabling IPv6 in lwIP causes internal APIs to shift from ip4_addr_t to the dual-stack ip_addr_t type. Any SDK glue code written for IPv4 using ip4_addr_t explicitly will fail to compile once IPv6 is enabled. The first compile error in this project from a type mismatch appeared in the DNS glue code of the Bouffalo SDK, exactly as Gupta et al. described. In terms of the replacement of ARP with interactive neighbor protocols such as neighbour discovery protocol (NDP) as defined in RFC 4861, there are material reasons for the way NDP works; specifically, there will be no broadcasting queries to each device connected to a link through NDP. Instead, the use of an ICMPv6 neighbour solicitation packet will be used to find devices that belong to the same solicited-node multicast group as the last 24 bits mathematically computed from the IPv6 address being examined. The only device that is assigned to the IPv6 address will reply to the neighbour advertisement, containing the corresponding MAC address. NDP (Neighbour Discovery Protocol), also handles the router discovery, prefix advertisement and duplicate detection of addresses using ICMPv6 messages. The

end result of the inability for devices on the link layer to generate their own neighbour cache via NDP will be that the cache will be empty, no MAC addresses for outbound IP traffic will be resolved, and there will be the appearance of routing errors; however, the errors actually occurring at the link layer, not the routing layer. According to RFC 9119, multicast delivery is not an automatic feature of 802 wireless networks, but rather dependent on how the driver has been programmed. For example, there are no known test cases on IPv6 EtherType frames when using a driver that has only been tested with IPv4. Therefore, without knowledge of the source code, one can never know how the driver will treat in this situation, including whether the driver builds the correct multicast MAC address for Neighbor Discovery Protocol (NDP) operations. Furthermore, the NXP developer community documented an actual case where a driver incorrectly built the multicast MAC address for NDP. This results in silent NDP failures across the deployment until the filtering logic in the driver was corrected. The SLAAC process begins when a machine brings up its upcoming IPv6 interface. The machine creates an EUI-64 address derived from the system's MAC address when it generates a new link-local address. However, the machine cannot begin using that link-local address until the duplicate address detection process has been completed. To complete the duplicate address detection process, the machine sends out a neighbor solicitation message and waits for a period of time determined by Retrans Timer, with default value set to 1 second as per RFC 4861. Once DAD has been completed, the link-local address can now be changed from TENTATIVE to VALID and used for sending and receiving packets through the network. lwIP implements the DAD properly and will not allow any packets to be transmitted from a link-local address unless it is in VALID state. In situations where the system boots up quickly and has highly constrained task timing, the one-second delay required for the duplicate address detection process can be easily missed by applications that attempt to send packets out shortly after IPv6 interface activation. Applications that attempt to send packets out from a link-local address in TENTATIVE state will receive an ERR_RTE response from lwIP, which will appear as no route could be found for the packet but is instead due to inability to send packets from an address in TENTATIVE state. While RFC 4429 provides for optimistic duplicate address detection, supporting the use of newly assigned link-local addresses prior to completion of duplicate address detection process, this capability is not provided by lwIP by default. The delay required by DAD is mandatory. Embedded networking does not always receive adequate attention when it comes to the problem of fragmenting memory.The FreeRTOS operating system supports five different methods of managing heaps; however, only two manage the coalescence of adjacent, released free blocks (i.e., they combine these blocks back together). If coalescence does not take place, then as the network stack allocates and frees memory (for example, when storing packet buffer space, protocol control information, neighbor cache entries and routing information for determining routes) over time, the heap will gradually become fragmented, making it likely that the next time that the memory management function pvPortMalloc() is called, it will return NULL to indicate there is no free memory available, even though the total amount of memory that is free still appears reasonable. That is, there are still free blocks of memory available, but none of them are adjacent, or is any single block large enough for the next allocation of memory. The FreeRTOS [9] community has also documented

the problem in terms of compatibility between the lwIP stack and FreeRTOS when using non-coalescing heap methods as a known issue. The problem is made worse when using an IPv6 network, because all of the various data that has to be stored for the purposes of working with IPv6 (e.g., maintaining neighbor cache entries, or maintaining IPv6 route tables, and SLAAC state machine entries/updates) and multicast membership tracking creates multiple independent patterns of dynamic allocation. The IETF 6LoWG states explicitly that driver level correctness is a significant requirement for IPv6 to work on constrained networks. In lwIP, a driver interacts with the stack using two function pointers in the netif structure: output for sending packets at the IP layer and link output for directly transmitting frames. The link output function in lwIP is called with an IPv6 frame. However, if a driver is written only for IPv4, it may set the wrong EtherType for an Ethernet frame representing an IPv6 packet or may fail to map the multicast address used by the NDP protocol onto the proper MAC format, or it may use IPv4 header length assumptions on a packet format that is invalid for an IPv6 packet. All of these possible errors would cause lwIP to receive an ERR_IF from the driver. The Bouffalo SDK STA-mode driver does not have any publicly available documentation about how to process IPv6 packets. The ERR_IFs seen during this project and the problems with the link-layer header indicate that the driver does not fully support transmission of IPv6-style packets, but confirming this conclusion will require either collecting packet data via an external Wi-Fi monitor or obtaining access to the driver's source code. Both of these needs are identified as future work in section 7.

4 System Architecture

4.1 Overview

The system is designed to establish the connection between two devices using IPv6, but there are no external devices such as routers, gateways, or Internet connectivity. It consists of two PineCone BL602 development boards, both of the same kind, but they are used for different purposes. One of them is used to act as a Wi-Fi Access Point, and the other one acts as a Wi-Fi Station. The main idea behind this is to establish a Wi-Fi network between these devices, and both of them are communicating only using IPv6. The system is organized into various layers such as, hardware, operating systems, networking, and applications. This makes it easier to find, configure, and enhance the system. Two devices are communicating using IPv6 UDP packets, and this helps to check the configuration of the IP addresses, routing, and packet transmission in the devices.

4.2 Hardware Architecture

In this project, we used PineCone BL602 in development board for the hardware platform. The development board uses a low-cost RISC-V microcontroller with combined Wi-Fi and Bluetooth capabilities, making the development board small and cost-effective for IoT development. It is limited in terms of memory and processing capabilities, and this is actually a limitation for testing embedded networking devices. The development boards

communicate to the host computer via USB, allowing power, flash, and debugging options. The built in Wi-Fi features of both devices allow for direct connection, eliminating the necessity for additional network interface hardware. Using the same development board for both the AP and STA activities leads to consistent system operations and enables device duplication and understanding.

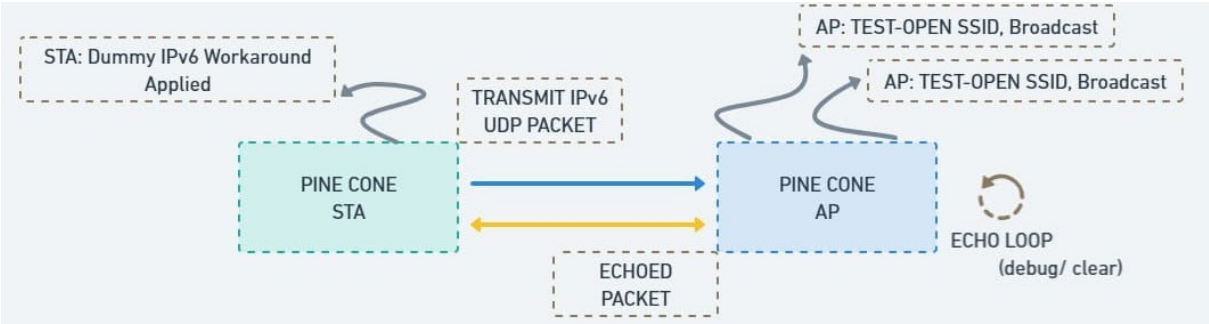

Figure 1: System architecture diagram.

4.3 Software Architecture

The software is structured in various layers that work together to provide networking functionality. These layers include the operating system, Wi-Fi management system, networking stack, and the application layer.

4.3.1 Operating System Layer

The firmware is based on a small real time operating system known as FreeRTOS. FreeRTOS is a common operating system that provides various services that are required when performing overlapping networking functions. Different tasks are created to provide Wi-Fi functionality, start up the networking stack, and lastly, run the application. This is done to avoid imbalance in the system because of networking functions.

4.3.2 Networking Stack Layer

The networking layer depends upon the Lightweight IP (lwIP) stack, which is included in the Bouffalo SDK. This lwIP stack includes IPv6 protocol, ICMPv6 messaging, UDP communication, and Neighbor Discovery. The network stack configuration includes IPv6 capabilities and methods for better network connection. The network layer supports IPv6 address and Stateless Address Autoconfiguration (SLAAC).These allow devices to connect to the network and automatically update their IPv6 addresses. The Neighbor Discovery Protocol (NDP) is enabled in the network stack. It supports in the recognition of neighbors and address resolving. ICMPv6 messages are turned on on the network layer. These messages help with reporting problems. UDP over IPv6 is configured in the network layer.The network layer supports UDP via IPv6. It helps in UDP based systems. The network interface (netif) layer of the lwIP connects a protocol layer to the Wi-Fi driver.

4.3.3 Wi-Fi Management subsystem

The Wi-Fi system allows wireless networks along with is implemented using a finite state machine (FSM). It consists of a few steps starting with the startup, scanning, authentication, association, and communication maintenance. The device functioning as an Access Point (AP) creates a network with a particular SSID. On another hand, the Station device searches for a connection and connects to the AP. Once the connection procedure is performed, the network connection interface is activated, and the IPv6 stack is started. Correct communication among Wi-Fi management and a network layer is important to the device's safety. It is important to handle event callback messages from the Wi-Fi component while stopping.

4.3.4 Application Layer

Application layer provides a simple UDP-based communication method to test and verify IPv6 protocol functions. The Access Point system process as a UDP server, redirecting incoming IPv6 packets back to their sender. The Station device acts as a UDP client, sending information to the server and waiting to receive answers. This technique tell the difference between network connections and applications, by that making it easier to find possible issues in an IPv6 based network.

4.4 Communication Flow and Data Path

The information path in the packet transmission process has various levels. Each and every layer offers a particular purpose for transmitting data. The Application Layer starts the procedure by way of generating a UDP message with user data for communication. The Transport Layer provides a UDP header to the UDP message. The UDP header provides the ports the authentication used in the procedure. In the Networking Layer, IPv6 protocol headers are added to UDP packets. The IPv6 header specifies where to send data to its destinations. At Link Layer, data is arranged for transfer through Wi-Fi.At the Physical Layer, data is communicated across Wi-Fi. At receiving end, the procedure is reversed. The whole process depends on proper coordination and interaction between all layers which are involved. Wrong configuration settings at any level may block data transfer over an entire network.

4.5 Design Consideration and Constraints

Some design considerations were taken into account for the architecture. The BL602 platform has limited memory. Secondly the dependency of the state of the Wi-Fi connection on the IPv4 configuration. There are Synchronization conditions have to be considered during the design process, as some of the components are event-driven. And the hardware must be kept to a minimum. The architecture must be simple and allow for the and systematic debugging of the IPv6. Taking all of these considerations into account, we designed a simple yet effective architecture with clear variables, as well as methodical IPv6 debugging.

5 Implementation

5.1 Development Setup and Firmware Structure

The procedure for implementation consisted of the creation of two distinct firmware based programs which were able to communicate in a similar hardware system (PineCone BL602) but developed for various runtime programming functions: one of them for the Access Point (AP) device and another for the Station (STA). Both programs were developed with the Bouffalo SDK with the lwIP package to handle the Internet Protocol layer. The boards were connected to a host computer via a USB connection for power supply and f lashing. The primary debugging tool employed during the implementation process was the serial log, considering that deep packet inspection and network sniffing were not possible during the initial stages of system development. In order to maintain testability within the architecture, the implementation chose to follow a layered methodology that consists of the following steps. The system's initiation and RTOS operating system stability features (booting procedure, task generation, heap access), wireless network subsystem's dependability (AP generation, STA connection, event processing), lwIP IPv6 stack-based enabling (netif IPv6 parameters, ND protocol addressing assistance, ICMPv6 protocol support, link-local support), and application-layer testing. Finally, IPv6 routing and neighbor resolution validation was also performed to validate that the UDP packets could be routed to the peer This was necessary because the enablement of IPv6 on the BL602 increased the memory consumption and also revealed synchronization issues within the Wi-Fi manager.

5.2 lwIP IPv6 Enablement and Configuration

IPv4 networking is turned on by default for most embedded SDKs. Code must be changed in order to enable IPv6 networking. We enabled IPv6 in lwIP by enabling IPv6 options and enabling the corresponding code for it to communicate. Some key features are that work with IPv6 include, **IPv6 core support:** Core data structures that deal with IPv6 address definitions and processing of IPv6 header data, as well as how the stack handles inputs and outputs of the stack for IPv6 protocol. **ICMPv6 support:** This is used for neighbor discovery protocol, which is essential for IPv6 support. **Neighbor Discovery Protocol support:** This includes neighbor cache, router solicitations and duplicate address detection, along with other functions and possibly according to current configuration. **UDP over IPv6 support:** This includes supporting the User Datagram Protocol over IPv6; socket connections, protocol control block connections and checksums support for IPv6. **Auto-configuration support:** This means supporting link-local addresses and providing stateless address auto-configuration support. The next thing to do once this has been enabled is to make sure the "netif" for the Wi-Fi connection has started properly within lwIP and is indicated as "up." This is vital for IPv6 since it relies on the netif to discover neighbors, prepare IPv6 packets and properly process them. Most of the routing issues that are seen later on with lwIP, specifically "no route" errors, are due to netif related problems such as not being able to reach a destination via any of the netifs, an absence or incorrectly configured default route within the netif, the neighbor

cache being empty meaning that NDP isn't complete and an incorrect scope or zone for link-local addresses, particularly when dealing with fe80::/10 addresses. Since the system was designed with support for the IPv6 protocol means it is also imperative to verify that the IPv6 netif is not being prevented from operating properly due to a presence of an established IPv4 connection in the firmware around it.

5.3 Access Point Firmware Implementation

5.3.1 AP Role and Responsibilities

The AP firmware was developed to implement three different functions. The initial method is to set up a Wi-Fi access point and transmit a well known SSID for all stations to identify and locate. The SSID is defined as TEST_OPEN. The following step is to set up IPv6 with the AP's Wi-Fi network interface with lwIP. The third approach is to build a UDP echo server that will accept, transmit back IPv6 UDP packets to their initial sender. The process that follows components allows the AP to function as a reliable and simple network destination for station (STA) testing. The echo server implementation is also important. The echo server returns again to the sender exactly the same information it received. Receiving the same information as provided verifies that all network elements are functioning efficiently.

5.3.2 Wi-Fi AP Startup and the Deadlock Fix

AP configuration was not stable in the first process stage of the integration process, even though log data proved that the AP start was executed. This happened, due to the AP was established during a Wi-Fi event callback. This is the major issue since the Wi-Fi manager used a different and separate event loop than the standard timing methods. The AP was not able to move or proceed due to a blockage resulting from a waiting event., The SSID was not visible on other devices, although there was a correct AP startup according to logs. The AP initialization was removed from the callback method. It was completed from a separate job. Once completed, the AP was properly initiated. This was an important point for the team as the task beyond cannot create a event network to testing the IPv6 technology.

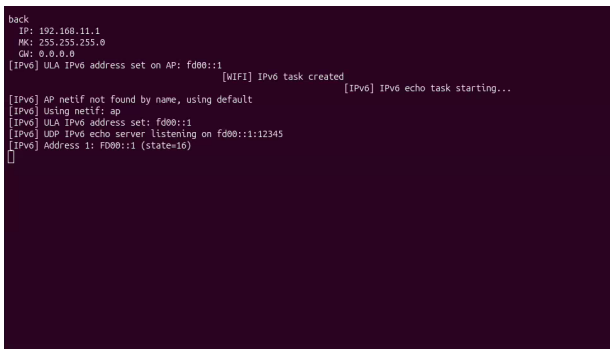

Figure 2: Ap Starting Point.

5.3.3 IPv6 Interface Bring-Up on AP

The goal of the next step was to configure the lwIP-based AP interface for IPv6 after successfully configuring the AP in AP mode. In order for the AP's interface to support IPv6, you will first need to have the Wi-Fi netif operational and running, then enable IPv6 support on this netif, and finally ensure that the AP can obtain at least a link-local IPv6 address. During this testing phase, serial logging was used to verify two very important items. One of the requirements for the final project goal is that there will not be any internet access via a router, therefore making the project goal local device-to-device connectivity only.

Figure 3: AP creating the IPv6 address.

5.4 UDP Echo Server (IPv6)

An application protocol consists of a User Datagram Protocol (UDP) echo server that utilizes an IPv6 socket. The UDP echo server provides the ability to implement the following functions. Create an IPv6 UDP socket and Bind the UDP socket to an available and defined UDP port number. Wait for datagrams to come into the uplink. When a datagram is received it immediately sends the received datagram back to the source address that sent it. The simplicity of the application allows you to debug it more easily if there is an error in the data sent, because if the application fails, it will be a network configuration issue, address issue, routing issue, neighbour discovery issue, or socket binding issue which makes it much less complicated than any part of the application itself.

5.5 Station Firmware Implementation

5.5.1 STA Role and Responsibilities

An STA is a firmware item whose function is to connect to the TEST_OPEN Access Point, and once connected, initialize the lwIP IPv6 network stack, create a UDP client socket, send data packets to the AP's IPv6 address, receive echoed data packets from that same address, and check the integrity of the received data. This is a standard IoT client way to connect to an IP network and send or receive packets to and from a service endpoint.

```
bssid[0] = 0xffff
bssid[1] = 0xffff
bssid[2] = 0xffff
search ssid = TEST_OPEN
result ssid index = 0
===end sm_get_bss_params===
AP Security mode: wep:0,wpa:0,wpa2:0,wpa3:0
Probe Response
timestamp = 0x81a2913
AP Security mode: wep:0,wpa:0,wpa2:0,wpa3:0
------ 269 set default key 0x42022300, ptr 0x00000000
ind ix 0x42023cac, chan_ctxt is 0x420242f4
[RX] Connection Status
[RX]   status_code 0
[RX]   connect result: sm connect ind ok
[RX]   MAC AC:D8:29:51:CE:53
[RX]   vif_idx 0
[RX]   ap_idx 0
[RX]   ch_idx 0
[RX]   qos 0
[RX]   acm 0
[RX]   assoc_req_ie_len 27
[RX]   assoc_rsp_ie_len 16
[RX]   aid 0
[RX]   band 0
[RX]   center_freq 2462
[RX]   width 0
[RX]   center_freq1 2462
[RX]   center_freq2 0
[WF][SM] Exiting connecting state
[WF][SM] State Action ###connecting### --->>> ###wifiConnected_ipObtaining###
[WF][SM] Entering wifiConnected_ipObtaining state
[WF][SM] DHCP Starting...0x42018cc0
[BL] [SEC] Failed after loop (17)
link_header size is 94 vs header 0
[WIFI] Connected
                [STA][IPv6] STA netif not found by name, uing default
                                                                [lwip] netif status callback
  IP: 0.0.0.0
  MK: 0.0.0.0
  GW: 0.0.0.0
 SKIP Notify for set Empty Address
[STA][IPv6] ULA IPv6 address set: fd00::2
                                        [lwip] netif status callback
  IP: 192.168.11.100
  MK: 255.255.255.0
  GW: 192.168.11.1
[WF][SM] Exiting wifiConnected_ipObtaining state
[WF][SM] IP GOT IP:192.168.11.100, MASK: 255.255.255.0, Gateway: 192.168.11.1, dns1: 0.0.0.0, dns2: 0.0.0.0
[WF][SM] State Action ###wifiConnected_ipObtaining### --->>> ###wifiConnected_IPOK###
[WF][SM] Entering wifiConnected_IPOK state
[WIFI] Dummy IPv4 injected
                        [WIFI] Network ready → starting UDP client
                                                                [UDP] Client task started
                                                                link_header size is 94 vs header
```

Figure 4: Starting point of STA where the IPv6 is connected.

5.5.2 Connection State Handling and IPv4 Dependency (Previous Work Applied)

The Wi-Fi manager considers connections established by completion of the IPv4 configuration as being valid. This means that the internal state machine, which is part of the firmware stack for the BL602 chipset assumes that there will be a valid and uninterrupted connected state, once an IPv4 address (such as from DHCP) is received. If for any reason, an STA is not able to complete the IPv4 configuration process, it is forcibly disconnected from the network, even if IPv6 is present and functioning at the time of disconnection. This is significantly different from how laptops and phones behave, where IPv6 is in a ready state and IPv4 has already been established. Therefore, in an IPv6-only configuration, this dependency on IPv4 creates a large architectural difficulty to overcome. To provide continued IPv6 testing and to work around this environment, a dummy IPv4

configuration was created with the requirement of the Wi-Fi manager FSM and to minimize the frequency of STA disconnections. With this form of injection, the STA can be connected to the network before the IPv6 communication (UDP packets) occurs between the client and STA. It is important to include this resolution as part of the report. Even the report describe the firmware limitation that will affect IPv4.

Figure 5: Data Packet issue.

6 Methodology

6.1 Research Methodology Overview

The aim of this project is to verify whether two small embedded devices (PineCone BL602) can talk to each other using the IPv6 protocol without the help of any external network devices. This project combines theory and practice. The methodology is divided into five major phases. The methodology involved the following five major phases: preparation of the system along with the establishment of base stability, enabling the IPv6 stack, configuring the Access Point (AP) along with the Station (STA), testing the communication using UDP over the IPv6 stack, and repeatedly isolating the faults. This methodology helped to identify the exact abstraction level causing the problem rather than the higher-level network components.

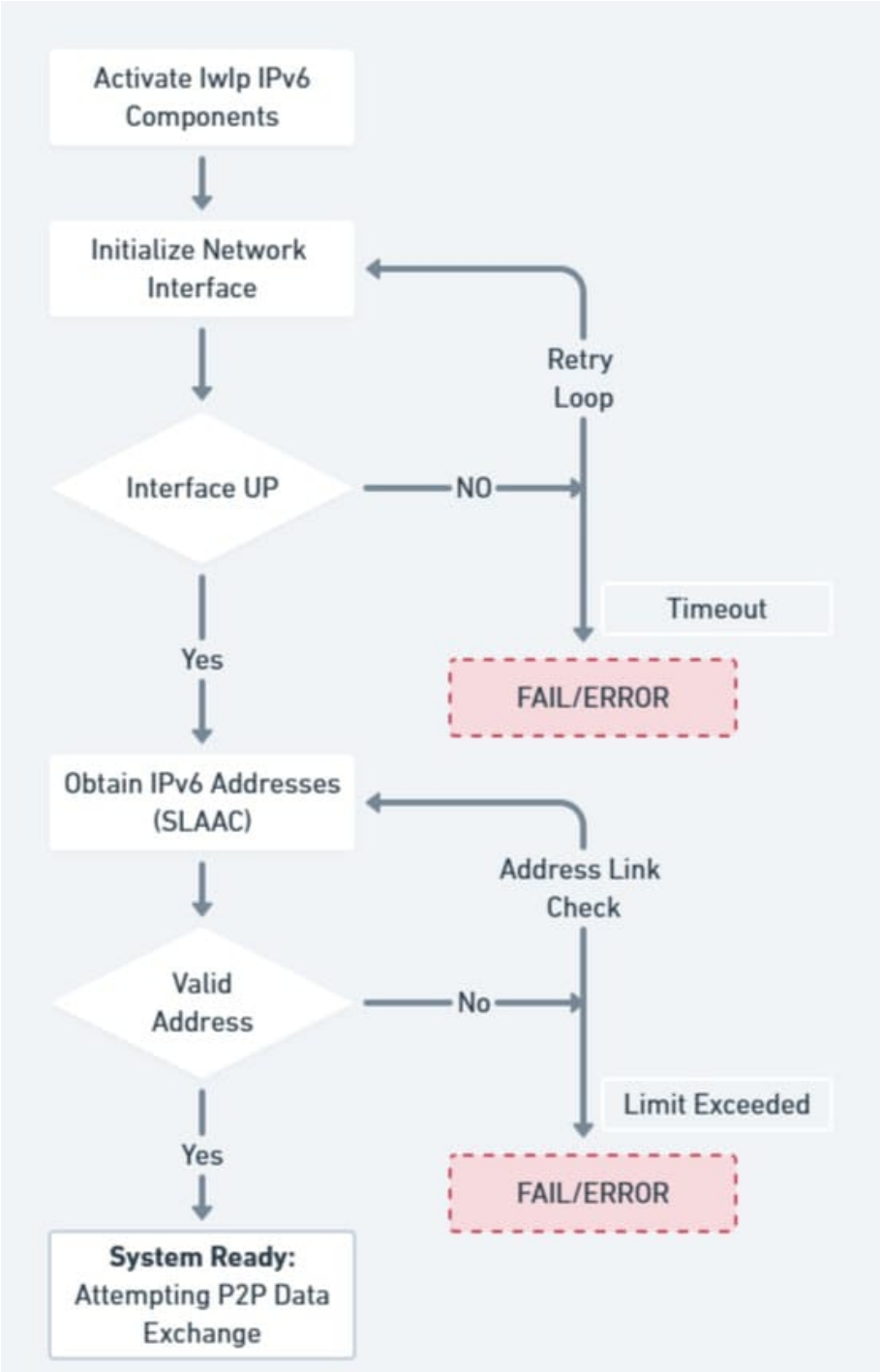

Figure 6: Work Flow of the system.

6.2 Layered Validation Approach

Layer 1: Hardware and RTOS Verification, Verify that the device can start successfully. Confirm that FreeRTOS tasks are running correctly, Ensure that your device is not

restarting unexpectedly. **Layer 2:** Wi-Fi Connectivity Verification. It Verifies that an Access Point has been created Confirm that a Station has been successfully associated with the Access Point and verify that the Access Point SSID is visible to potential stations. **Layer 3:** IPv6 Stack Verification, Ensure that IPv6 has been enabled within the lwIP stack. Confirm that a link local address has been generated on the network interface. Verify that the status of the netif network interface is up. **Layer 4:** Application Layer Verification, Start a Unified Data Protocol (UDP) echo Server on the Access Point (AP) Create a UDP Client on the Station (STA) Attempt to send an IPv6 Packet from Station (STA) to Access Point (AP). Instead of moving to Layer 2, Layer 1 must be validated. During Validation, if unstable conditions are encountered in the Validation process, the methodology specifies that the last validated layer will need to be revalidated.

6.3 Diagnostic Tools and Techniques

Since it was an embedded system, Serial logging was primarily used for observing system behavior. lwIP debug flags during compilation were also used for obtaining debug information. Runtime inspection of interface states was also done for obtaining real-time information on system functionality and Observing the changes to the connection state machine. Since packet sniffing was not possible, the focus was on, Observing lwIP return codes and changes to the connection state machine also looking into the error messages like routing failures.

7 Debugging and Problem Solving

7.1 Phase 1: System Instability Due to Heap Exhaustion

When we started using IPv6, we had many issues with our system not functioning properly after we installed support for this new protocol. When the devices would start up or attempt to start their network stacks (their internal network connections) they would either restart completely or fail to initialize their network stacks. In addition, from the serial logs, we discovered that all of the devices had run out of memory to use, which meant that there was enough memory allocated to IPv6 that it exceeded which was default for FreeRTOS. The extra memory allocated for IPv6 was primarily due to implementing the IPv6 neighbor cache, ICMPv6, multicast, and routing functionality. This was caused by the overall memory allocation to devices while the startup tries to exceed how many memory's physical availability on the BL602. To resolve this issue we increased the existing FreeRTOS heap allocation so that there would be enough memory for devices to use after they boot up and prior to receiving IPv6 support. Once we resolved the memory issue, we were finally able to get all systems to start up correctly and boot up without issue, which was necessary in order to move forward with the rest of our troubleshooting.

7.2 Phase 2: Wi-Fi Access Point Deadlock

Later on, once the stability of memory was attained, there occurred an one more issue. This Issue was related with the development of an Access Point (AP). The firmware had recorded the issuance of the AP initialize command. However, the SSID was not detectable and can be directed on the exterior. Also, it was detected that the system was not responsive after initialization. The effective analysis of the code showed that the AP creation of some code had been executed as a Wi-Fi event callback. The code for the Wi-Fi manager was a state machine that was internally synchronized. Therefore, a translation into a major state had been implemented in the callback context. This had led to an deadlock. It was discovered that there was a blocking state with the firmware. This had developed the activation of the AP. The corrective measure was restructuring the code to perform the AP initialization when one is not in the callback context. The AP initialization was forced to be performed as an activity. Once the code was fixed, the SSID had been found to be detectable. This had established that the issue had been hold responsible for the architecture.

7.3 Phase 3: Station Disconnection Due to IPv4 Dependency

Attention was turned to the Station device after the Access Point was estabilised. The Station was able to connect to the AP in the tests done on IPv6-only connectivity, but then it was immediately disconnected. This disconnection was due to a requirement that the Wi-Fi manager must have a functioning IPv4 setup prior to being able to fully trust the connection. This requirement was a firmware requirement because of the lack of IPv4 in the tests of IPv6-only connectivity. This requirement became a problem because the tests for IPv6-only connectivity were occurring at that time. To correct this problem, a temporary fix was placed in order to inject an IPv4 configuration placeholder after the association. By adding this placeholder, the tests for IPv6-only connectivity were able to continue without being disconnected from the AP. The only issue with this is that it is a temporary solution for the firmware requirement of having an IPv4 present before the connection would be able to be totally trusted across the network. As stated above, this requirement created a problem during the testing of the IPv6-only connectivity, which was being conducted at that time. So, the temporary fix will create ongoing issues with the firmware for the time being due to the requirement of having an IPv4 present prior to building any trust with the connection over the network.

7.4 Phase 4: IPv6 UDP Routing Failure

After connecting to stable Wi-Fi and initializing IPv6 on both devices, we attempted using UDP to communicate over the IPv6 network. Both devices had valid link-local IPv6 addresses and there were no problems with creating UDP sockets. We did experience problems sending data due to errors at the routing layer of lwIP. The echo response was missing, thus confirming that the overall instability in the system at the network layer has now been resolved to a more localized level within the network protocol, and that both devices are now stabilizing. Earlier, there were also several problems limited down more

specifically to the newer established Wi-Fi interface and the newer established structured debugging process.

7.5 Engineering Insights from the Debugging Process

The process of debugging and problem-solving helped gain several engineering insights. The process revealed that enabling IPv6 on embedded systems is not just a simple configuration issue but also involves architecture-level considerations, memory management, and integration of existing firmware components. The process also highlighted that when designing an event-driven system, it is important to avoid state changes within callback functions, which could cause deadlock situations. The process also revealed firmware-level biases towards IPv4 connectivity, which could be a barrier when working exclusively with IPv6 connectivity. Perhaps the most important insight gained is that this systematic approach has helped transform an unstable system into a stable experimental system where problems are well contained. While it is not possible to send UDP packets over IPv6, this process has helped identify that the problem lies in one of the layers of the routing or neighbor resolution process.

8 Results and Evaluation

The switch of IPv6 consumes more memory compared to the switch of IPv4 only. The reason is that IPv6 requires additional memory in neighbor cache, multicast group, ICMPv6 task and bigger packet buffers. The additional memory remains within harmless boundaries following the arrangement of the stack. There were no poor performances of Wi-Fi after the system stabilized. This indicates that IPv6 does not reduce the speed of the wireless connection in case it is properly configured, still it requires more memory. It has been tested that IPv6 can be turned on the PineCone BL602 board, although it requires some careful planning and firmware alteration. IPv6 is also enabled and Wi-Fi is okay. The issue that is yet to be resolved explains the difficulty of incorporating the IPv6 with Wi-Fi driver in this board. The layer that requires improvement to make in future has been identified by debugging. Though the IPv6 UDP communication is yet to be completed, we have a good foundation to develop on other embedded systems which utilize IPv6.

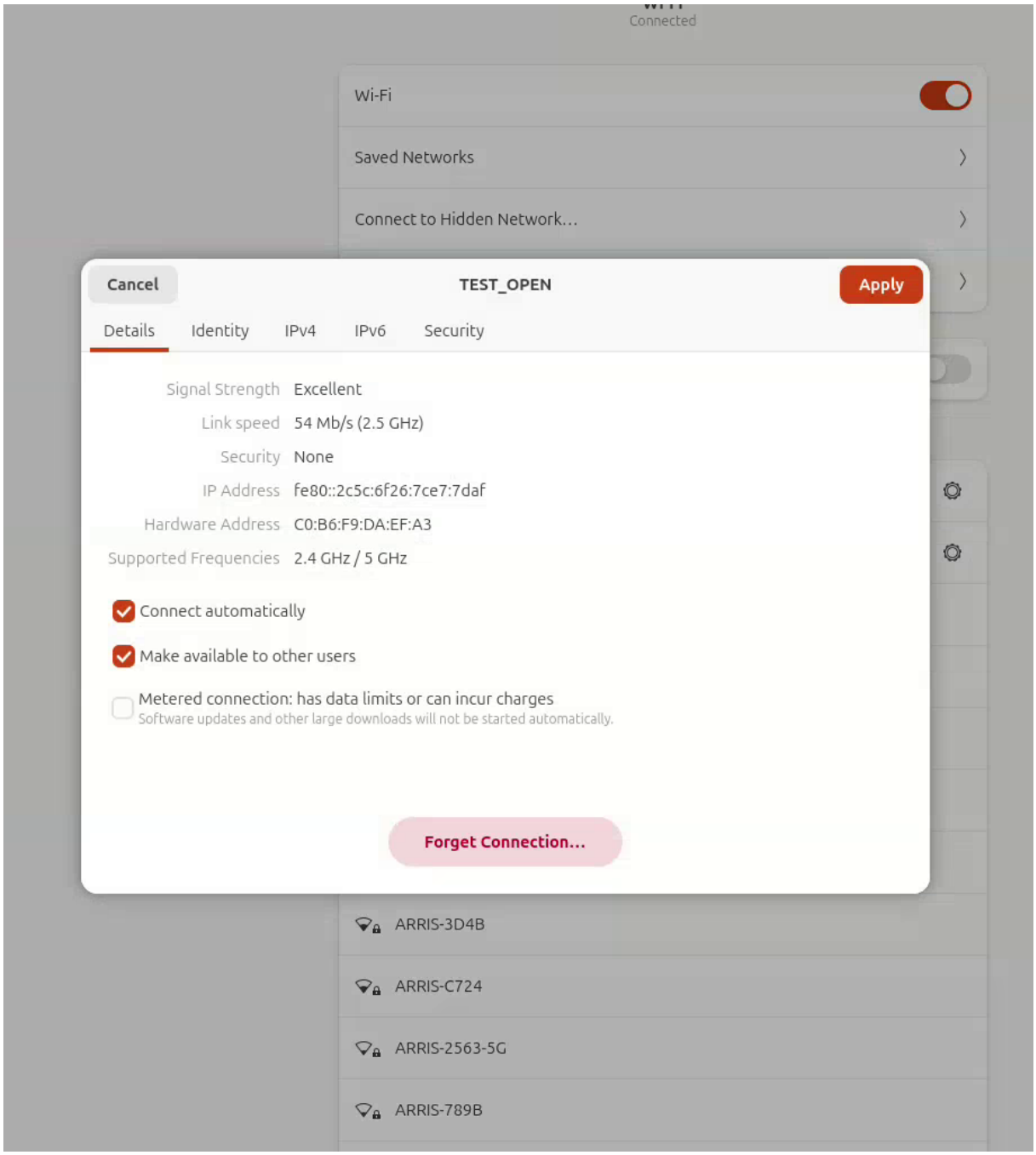

Figure 7: Laptop connected with AP and IPv6 is Assigned.

9 Discussion

Neighbor Discovery Protocol (NDP), replaces the ARP (Address Resolution Protocol) found in IPv4. The purpose of NDP is to convert IPv6 addresses into MAC (Media Access Control) addresses which are used at Layer 2 (Link Layer). When a device has a packet it wants to send, it first sends a message known as a Neighbor to discover the

MAC address of the intended destination device. If there is a response from the other device containing its MAC address (the Neighbor Advertisement), then the packet can be sent using this MAC address. In the event that there were no successful discoveries of the MAC address of the other device, then the Neighbor Cache would not have been populated with the information needed for sending the packet. This process relies heavily on multicast and Link Layer processing. Therefore, if there is an issue with the way In the event this occurs, then the packet transmission will not be successful. One of the other likely causes could stem from the scope of link-local addresses, which are limited to a single Network Interface (e.g. Ethernet). In order to send packets utilizing a link-local IPv6 Address, the networking stack must have correctly associated the link-local address with the sending Network Interface during routing. If this is not possible for any reason, the Network Stack will not recognize that there is a valid Network Interface available to use when sending the packets. Thus, resulting in data transmission failure. The logs of the UART show that the IPv6 addresses have been correctly allocated from the DHCPv6 server. However, the packet transmission could not occur due to a routing failure. A second possible issue is due to the interaction between lwIP and the Wi-Fi driver. As can be seen in section 8.4, the lwIP stack is responsible for creating the IPv6 packets, which are sent from lwIP to the interface layer. The Wi-Fi driver is then responsible for creating a data link layer frame and transmitting it wirelessly. This will complete the full transfer of a packet (as explained in section 8.4). It is very difficult to perform any analysis of this final step because the Bouffalo SDK Wi-Fi firmware binary files are pre-compiled. The major issue for developers of IPv6 for the BL602 is likely to occur due to the fact that the coding of the driver that links this stack with the hardware is not open source software, which makes it difficult for research and testing of this operation with Wi-Fi. Future work must be done in this area for successful implementations of full IPv6 communications on the BL602 platform.

10 Future Scope

10.1 Advanced IPv6 Routing and Neighbor Discovery Analysis

Tests performed in the future need to focus primarily on testing the functional aspects of both the IPv6 routing protocols and how lwIP implements its Interaction with the IPv6 routing protocols, as well as the operation of the neighbor discovery protocol (NDP). A current test of the IPv6 network shows that the lwIP stack is initializing correctly and assigning the correct address; however, UDP packets are still unable to reach their destination due to either a failure in routing to their destination or a failure in determining the MAC address of the destination. Future tests should verify what is in the IPv6 routing table, which network interfaces are being used, and how frequently the entries in the neighbor cache are used in order to determine if the failure to communicate occurs prior to mac address resolution (link layer) or after the mac address resolution occurred.

10.2 Deeper Protocol-Level Tracing Beyond UART Logging

This project involves the considerable reliance on UART log debugging as well as lwIP return code debugging since less detailed access was available through the APIs. Future enhancements should include providing access to additional debug capabilities in lwIP such as having the ability to monitor protocol-level debugging and using external packet monitoring tools that can be used in Wi-Fi monitor mode will allow us to see any IPv6 packets transmitted over the wireless channel and will aid in determining where any packet fails; whether it failed prior to being sent or whether it failed at the driver layer level.

10.3 Investigation of Interface Binding and Address Scope Handling

Researching how lwIP's implementation of the lwIP stack selects an interface to communicate with would be another possible area of research, as well as determining how link-local IPv6 has a limited scope due to its dependence on the network interface for link-local communication, in proper sequence this will determine the causes of the failure to route packets based on the link-local IPv6's not being communicated properly. Reason for the packet not being able to be sent would include lack of an interface binding.

10.4 Removal of IPv4 Dependency in the Wi-Fi Manager

A major issue that was identified during its development was that the Wi-Fi state machine relies on a successful IPv4 setup. This means that even if IPv6 networking is fully ready, the firmware would not be ready until it has an IPv4 address. In future, this state machine needs to be redesigned or adjusted so that simply being ready with IPv6 alone is a valid state. This would allow the BL602 platform to be used in a true IPv6-only network environment.

One of the principal problems recognized while developing the product is that the Wi-Fi State Machine depends on an operating IPv4 to function properly, even if IPv6 is fully functional on the hardware under development, the firmware will still need an operating IPv4 in order to function as intended. In future designs, the Wi-Fi State Machine will need to be redesigned so that being an operating IPv6 node can also be considered as a valid state. This will enable the BL602 to be used in a valid IPv6 only Network environment.

10.5 Driver-Level Verification of IPv6 Frame Transmission

The last and most important step is to determine how the Wi-Fi driver sends IPv6 frames. Since the Wi-Fi driver is closed source in Bouffalo SDK, it is difficult to determine if IPv6 frames are correctly sent at the link layer. This could be part of the future work, where we could capture Wi-Fi packets and reverse engineer how they are sent, or contact the SDK authors to determine how IPv6 frames are sent. This will determine if the last failure is in the driver or somewhere else in the networking stack.

The final and the most critical step is to define the way that the Wi-Fi driver transmits IPv6 frames. As the Wi-Fi driver is a closed source application in Bouffalo SDK, it is hard to know whether IPv6 is present. Correction of the transmission of frames is done

at the link layer. This may end up in the future employment where we may interrupt the Wi-Fi packets and reverse the way they are transferred, or contact the SDK. This will be in determining whether the last failure is in the driver or in some other portion of the networking stack

11 Conclusion

This report provides an excellent basis for performing technical evaluations of IPv6 capabilities on the PineCone BL602 (a low-cost IoT board) in establishing an infrastructure for supporting IPv6 protocols on inexpensive embedded devices. We will demonstrate our ability to create and use IPv6 on a low-cost embedded IoT device and highlight the challenges related to the design and implementation of IPv6 protocols on such devices. Not only is the experience from this project applicable to the PineCone BL602, but the findings will serve as a useful foundation for future research in the area of embedded networking. Along with presenting specific results, this report will give information on more general challenges to supporting IPv6 in large quantities of embedded IoT devices. In fact, this study illustrates that just because we enable a protocol does not mean we can effectively deploy it; we must also have good systems architecture, memory management, and protocols stacks in order to deploy and utilize protocols like IPv6 successfully. The results of this study demonstrate that many firmware stacks designed to operate on embedded devices utilize an IPv4 network architecture and therefore present challenges when implementing pure IPv6-based networking. Additionally, by clearly identifying and addressing system-level issues, this work will help to further the deployment of IPv6 on small, low-power IOT devices, as well as help illustrate the practicalities of operating current Internet protocols on an embedded device.

12 Review and Remarks from Authors of Other Teams

Table 1: Reviewer Comments and Author Responses

No.	Reviewer Comment	Decision	Author Response / Changes Made
1	The routerless architecture description is misleading because the Access Point technically acts as a router.	Addressed	The revised report clarifies the experimental topology where one PineCone operates as the Access Point and the other as the Station. The AP serves as the communication endpoint within a controlled test environment rather than a full router.
2	The paper does not clearly explain what changes were made in lwIP configuration to enable IPv6.	Addressed	The implementation section now explains enabling IPv6 features such as Stateless Address Auto Configuration (SLAAC), Neighbor Discovery, and link-local address support in the lwIP stack.
3	The paper lacks reproducibility because configuration changes and system parameters are not described.	Addressed	The final report provides a clearer description of firmware setup, network stack initialization, and the role of both AP and STA firmware modules.
4	The literature review does not clearly connect to the project topic.	Addressed	The literature review was rewritten to explain IPv6 fundamentals, address exhaustion in IPv4, and the need for IPv6 adoption in IoT systems.
5	The background section includes topics that are not directly relevant to the project.	Addressed	The literature review was refined to focus on IPv6 networking, embedded IoT platforms, and protocol transition challenges.
6	The results section lacks technical depth and does not explain the encountered problems clearly.	Addressed	The results and discussion sections now describe debugging observations including routing failures, Neighbor Discovery issues, and packet transmission problems.
7	The developed solutions are not clearly explained.	Addressed	The report now documents the fixes applied during development, including heap adjustments, resolving Wi-Fi manager deadlocks, and implementing the IPv4 workaround required for connection stability.

No.	Reviewer Comment	Decision	Author Response / Changes Made
8	It remains unclear whether IPv6 communication actually works.	Addressed	The revised report clarifies that IPv6 initialization and address assignment work correctly, but IPv6 UDP packet exchange remains unresolved due to routing and neighbor discovery issues.
9	The dummy IPv4 configuration workaround is unclear.	Addressed	The report now explains that the Wi-Fi manager requires IPv4 configuration before entering a stable connection state, which required the use of a dummy IPv4 configuration during testing.
10	The paper exaggerates achievements while the main goal (IPv6 UDP communication) is not achieved.	Addressed	The revised report clearly states the achieved results and explicitly identifies the remaining issues preventing full IPv6 UDP communication.
11	The abstract is inconsistent with the actual results.	Addressed	The abstract was rewritten to accurately reflect the scope of the work and clarify that full IPv6 data exchange is still under investigation.
12	The conclusion claims that final steps are clear, but the paper does not explain them properly.	Addressed	The conclusion was revised to summarize the achieved progress and identify future work required to complete IPv6 communication on the platform.
13	The paper lacks high-quality references.	Addressed	Additional academic references and RFC documents related to IPv6 networking and IoT systems were added to strengthen the literature review.
14	Some references may not be reliable.	Addressed	References were revised to include authoritative sources such as RFC documents and well-known networking publications.
15	Grammar and spelling errors make parts of the paper difficult to understand.	Addressed	The final report was proofread and edited to improve grammar, readability, and academic writing style.
16	The paper is relatively short and lacks detailed explanation.	Addressed	The final report includes expanded sections on implementation, debugging methodology, system architecture, and experimental observations.

No.	Reviewer Comment	Decision	Author Response / Changes Made
17	The background section contains redundant information.	Addressed	Redundant explanations were removed and the background section was reorganized for clarity.
18	The project appears unfinished because UDP communication is still failing.	Clarified	The revised report explicitly states that the project currently demonstrates IPv6 enablement and debugging progress rather than a fully completed networking solution.
19	Memory usage issues such as heap exhaustion are not quantified.	Partially Addressed	The report discusses the memory constraints and heap adjustments required to enable IPv6 functionality on the BL602 platform.
20	Wi-Fi manager deadlock behavior should be explained in more detail.	Addressed	The report now explains how the AP startup deadlock occurred due to event callback execution and how moving initialization outside the callback resolved the issue.
21	Routing errors during IPv6 communication are not sufficiently explained.	Addressed	The discussion section now examines possible causes including neighbor discovery failures, incorrect interface binding, and driver-level limitations.
22	The paper does not explain how the neighbor discovery process affects packet transmission.	Addressed	The report now explains the NDP process and how incomplete neighbor cache entries prevent packet transmission.
23	The relationship between lwIP and the Wi-Fi driver is not explained.	Addressed	The discussion now describes how lwIP constructs IPv6 packets and passes them to the Wi-Fi driver, which performs the final transmission at the link layer.
24	The paper should better identify the actual layer where the failure occurs.	Addressed	The report identifies the most likely failure point at the interface between the lwIP networking stack and the closed-source Wi-Fi driver.
25	The overall writing style could be improved and the paper should be more concise.	Addressed	The document was reorganized and edited to improve readability and maintain a clearer technical narrative.

References

[1] Mohsen Shirali et al. "A Survey on Anonymous Communication Systems with a Focus on Dining Cryptographers Networks". In: *IEEE Access* (Jan. 2023). DOI: 10.1109/ACCESS.2023.3242870.

[2] Tobias Tefke and Ralf C. Staudemeyer. "Sicherheit in Microservice-Umgebungen". In: *Sicherheit in vernetzten Systemen. 30. DFN-Konferenz*. Ed. by Albrecht Ude. DFN-CERT. Books on Demand, Feb. 2023, pp. D1–D21. ISBN: 9-783756-881390.

[3] Steve Deering and Robert Hinden. *Internet Protocol, Version 6 (IPv6) Specification*. RFC 8200. Internet Engineering Task Force (IETF), 2017. URL: https://datatracker.ietf.org/doc/html/rfc8200.

[4] Thomas Narten et al. *Neighbor Discovery for IP Version 6 (IPv6)*. RFC 4861. Internet Engineering Task Force (IETF), 2007. URL: https://datatracker.ietf.org/doc/html/rfc4861.

[5] Susan Thomson, Thomas Narten, and Tatuya Jinmei. *IPv6 Stateless Address Autoconfiguration*. RFC 4862. Internet Engineering Task Force (IETF), 2007. URL: https://datatracker.ietf.org/doc/html/rfc4862.

[6] Adam Dunkels. *Design and Implementation of the lwIP TCP/IP Stack*. Tech. rep. Swedish Institute of Computer Science, 2001. URL: https://savannah.nongnu.org/projects/lwip/.

[7] R. Gupta, P. Kumar, and S. Singh. "Evaluation of IPv6 Support in Lightweight TCP/IP Stack for Embedded Systems". In: *International Journal of Computer Applications* 145.12 (2016), pp. 15–21.

[8] J. Wu et al. "Transition from IPv4 to IPv6: A State-of-the-Art Survey". In: *IEEE Communications Surveys & Tutorials* 15.3 (2013), pp. 1407–1424.

[9] Amazon Web Services. *FreeRTOS Real-Time Operating System*. 2023. URL: https://www.freertos.org.

[10] lwIP Development Team. *lwIP TCP/IP Stack Documentation*. 2023. URL: https://www.nongnu.org/lwip/.

[11] Bouffalo Lab. *BL602 RISC-V Wi-Fi SoC Datasheet*. 2021. URL: https://bouffalolab.com.